technica engineering

Member of KPIT Group

Enhanced Ethernet Switch

Remote Control API

USER MANUAL

May 2024

Manual-Version: 1.0 FW-Version: v014.002 HW-Version: v2.0-v3.2

CONTENT

1 About this document 4
1.1 Scope 4
1.2 Validity 4
1.3 Glossary 5
2 Remote control API 7
2.1 HTTP Get Methods 7
2.1.1 Return the current configuration 7
2.1.2 Return the default configuration 7
2.1.3 Return the equipment information 8
2.1.4 Return the status information 8
2.2 Post Method 8
2.2.1 Validate a configuration and fix it if possible 8
2.2.2 Save configuration 9
2.2.2.1 Write the whole configuration in a persistent way 9
2.2.2.2 Write part of the configuration in a persistent way 9
2.2.2.3 Write a whole configuration in a non-persistent way 9
2.2.2.4 Write a configuration in a non-persistent way 9
2.2.3 Action commands 9
2.2.3.1 Save the current configuration stored into the NVM 10
2.2.3.2 Cold reset 10
2.2.3.3 Warm reset 10
2.2.3.4 Jump to bootloader mode 10
2.2.3.5 Set the EES in sleep mode 10
2.2.3.6 Alive command 10
2.2.4 On-the-fly commands 11
2.2.4.1 Address Table 11
2.2.4.2 Ports 12
2.2.4.3 PTP reconfiguration 13
2.2.4.4 MKA keys 14
3 Examples of use 15
3.1 Curl 15
3.1.1 Retrieve information 15

3.1.2 Set the configuration 16
3.2 Python 16
4 Changelog 18
5 Contact 19

1 About this document

1.1 Scope

This document describes the Enhanced Ethernet Switch's remote Application Programming Interface (API). All commands use standard HTTP request to retrieve or set the configuration of the switch.

Troubleshooting is not part of the scope of this document. For support queries, please contact Technica Engineering via support@technica-engineering.de

Please note that, due to the dynamic development of the Enhanced Ethernet Switch, the information in this document is subject to change without prior notice to adapt to technical changes and updates.

Make sure to always use the correct version of this document, matching the firmware and hardware versions in operation (see next section). For the latest version of this document please check the customer portal.

©2021 Technica Engineering GmbH

The content in this document is copyrighted by Technica Engineering GmbH. Copying or reproducing its content in full or in parts is not allowed. Technical contents in this document are subjected to change without prior notice. Third-party contents and trademarks are the property of the corresponding vendors.

Additionally, the following symbols are used throughout the document to highlight important information:

 Information: Useful tips, hints, or background information.

 Warning: Important information to consider for the correct use of the device.

1.2 Validity

This document applies to Enhanced Ethernet Switch with:

- Firmware version: v014.002
- Hardware version:
  - Up to v2.1 for TE-2002
  - Up to v3.2 for all other variant

1.3 Glossary

Term	Definition / Explanation
AVB	Audio Video Bridging
AVTP	(Layer 2) Audio-Video Transport Protocol
DUT	Device Under Test
EES	Enhanced Ethernet Switch
Eth	Ethernet
FCS	Frame Check Sequence. Error-detecting code is added at the end of an Ethernet frame
gPTP	Generic Precision Time Protocol, IEEE 802.1AS
HW	Hardware
IVN	In-Vehicle Network
IFG	The interframe gap between two ethernet frames
PTP	Precision Time Protocol, IEEE 1588
TECMP	Technically Enhanced Capture Module Protocol, used for encapsulation of captured in-vehicle network data
TSN	Time-Sensitive Networking
API	Application Programming Interface
MKA	MACsec Key Agreement
EAP	Extensible Authentication Protocol
CA	Secure Connectivity Association
Curl	Command is a tool used to transfer data from or to a server using various network protocols
MSK	Primary Session Key
NVM	Non-Volatile Memory
CAK	Connectivity Association Key
CKN	Connectivity Association Key Name
ICK	Integrity Check Key
KEK	Key Encryption Key
SAK	Secure Association Key
KS	Key Server

Table 1-1: Glossary

Additionally, the following symbols are used throughout the document to highlight important information:



Information: Useful tips, hints, or background information.



Warning: Important information to consider for the correct use of the device.

2 Remote control API

The Enhanced Ethernet switch has a remote-control API based on HTTP protocol. Two types of methods are used for configuration.

- GET Methods: These methods allow retrieving the configuration and status. Both json and msgpack formats are supported.
- Post Methods: These methods allow to modify the configuration and trigger some actions. These methods only support the msgpack format.

2.1 HTTP Get Methods

HTTP GET methods allow to retrieve the configuration and status using the json and msgpack format.

2.1.1 Return the current configuration

This command retrieves the current EES configuration

- *URL/api/get/config.msgpack* returns the complete configuration encoded in msgpack format.
- *URL/api/get/config.json* returns the complete configuration encoded in json format.
- *URL/api/get/config* returns the complete configuration encoded in json format.
- *URL/api/get/config.json/PATH* returns only the configuration contained in the specified PATH encoded in json format.
- *URL/api/get/config/PATH* returns only the configuration contained in the specified PATH encoded in json format.

For example: *URL/api/get/config/ports/main/0* returns only the configuration of the main port 0

For more information about msgpack format please check CHAPTER 2.2

2.1.2 Return the default configuration

This command retrieves the default EES configuration.

- *URL/api/get/default_config.msgpack* returns a complete default configuration encoded in msgpack format.
- *URL/api/get/config_default.json* returns a complete default configuration encoded in json format.
- *URL/api/get/config_default* returns a complete default configuration encoded in json format.
- *URL/api/get/config_default.json/PATH* returns only the default configuration data contained in the specified PATH encoded in json format.
- *URL/api/get/config_default/PATH* returns only the default configuration data contained in the specified PATH encoded in json format.

2.1.3 Return the equipment information

- *URL/api/get/equipment.msgpack* returns the complete equipment information encoded in msgpack format.
- *URL/api/get/equipment.json* returns the complete equipment information encoded in json format.
- *URL/api/get/equipment* returns the complete equipment information encoded in json format.
- *URL/api/get/equipment.json/PATH* returns only the equipment information contained in the specified PATH encoded in json format.
- *URL/api/get/equipment/PATH* returns only the equipment information contained in the specified PATH encoded in json format.

2.1.4 Return the status information

- *URL/api/get/status.msgpack* returns all the status information encoded in msgpack format.
- *URL/api/get/status.json* returns all the status information encoded in json format.
- *URL/api/get/status* returns all the status information encoded in json format.
- *URL/api/get/status.json/PATH* returns only the status contained in the specified PATH encoded in json format.
- *URL/api/get/status/PATH* returns only the status contained in the specified PATH encoded in json format.

2.2 Post Method

HTTP POST requests allow to set the configuration coded using the MessagePack format. MessagePack is an efficient binary serialization format, it is like JSON format but more compact.

2.2.1 Validate a configuration and fix it if possible

URL/api/validate/config.msgpack receives a EES configuration coded with the msgpack format and:

- Checks if all fields are within the limits. If any field is out of its limits the configuration is rejected.
- Any missing fields are added and filled with their default value.
- Any field not defined in the EES configuration is discarded.

The configuration is sent back corrected or an error indication if configuration is not valid and cannot be fixed. The sent configuration is neither saved in RAM nor in FLASH. This request can be used to migrate a configuration from an older version to a newer one.

2.2.2 Save configuration

Save configuration requests allow to modify and save the EES configuration.

2.2.2.1 Write the whole configuration in a persistent way

URL/api/set/strict/config_persistent.msgpack The sent configuration is saved into the NVM (Non-Volatile Memory). Therefore, the configuration is retained in case of a reset or loss of power supply. The sent configuration must fully comply with the structure used to define the EES configuration (no missing fields, no extra fields). For the configuration to become effective it is necessary to send a cold reset command or to turn off/on the power supply.

2.2.2.2 Write part of the configuration in a persistent way

URL/api/set/partial/config_persistent.msgpack The sent configuration is saved into the NVM (Non-Volatile Memory). Therefore the configuration is retained in case of a reset or loss of power supply. The sent configuration may contain a subset of the EES configuration, parameters do not present on the sent data are not modified. For the configuration to become effective it is necessary to send a cold reset command or to turn off/on the power supply.

2.2.2.3 Write a whole configuration in a non-persistent way

URL/api/set/strict/config_non_persistent.msgpack The sent configuration is saved into the RAM. Therefore this configuration will be lost in case of a cold restart or a power off/on. The sent configuration must fully comply with the structure used to define the EES configuration (no missing fields, no extra fields). To apply the configuration is necessary to send a warm reset command.

2.2.2.4 Write a configuration in a non-persistent way

URL/api/set/partial/config_non_persistent.msgpack The submitted configuration is saved into the device. Therefore this configuration will be lost in case of a cold restart or a power off/on. The sent configuration may contain a subset of the EES configuration, parameters not present on the sent data are not modified. To apply the configuration is necessary to send a warm reset command.

2.2.3 Action commands

Action command requests execute specific actions in the EES.

2.2.3.1 Save the current configuration stored into the NVM

URL/api/set/partial/config_non_persistent.msgpack saves the current configuration stored in RAM into the NVM.

2.2.3.2 Cold reset

URL/api/cmd/reset_cold discards the current configuration stored in RAM and restarts the device. At boot time, the configuration is taken from the NVM. If the configuration was modified before this request, the boot process may take some time to process the new configuration and save it in a special area to avoid this process on subsequent reboots if there are no changes to the configuration.

The EES WEB application uses this command along with *api/set/config.msgpack* to save and apply the configuration. The "SAVE and APPLY" button saves the configuration using *api/set/config.msgpack* and later sends *api/cmd/reset_cold* to activate the configuration. To check when the "SAVE and APPLY" has finished the EES WEB application uses the *api/cmd/alive* command to poll when the EES is ready again.

2.2.3.3 Warm reset

URL/api/cmd/reset_warm saves the current configuration in a dedicated RAM area that is retained after reboot and then restarts the device. On boot, the configuration is taken from that RAM, processed, and applied to into the device. The processed configuration is not saved, so it's faster than a cold reset but the configuration is not persistent.

2.2.3.4 Jump to bootloader mode

URL/api/cmd/bootloader discards the current configuration and jump to the bootloader program.

2.2.3.5 Set the EES in sleep mode

URL/api/cmd/sleep discards the current configuration and jump to sleep mode. Once in sleep mode the EES can only be woken by wake-up lines or bus CAN activity (functionality not implemented yet).

2.2.3.6 Alive command

URL/api/cmd/alive allows to check if the device is able to accept HTTP requests. An 8-digit number must be sent along with the alive command request. The alive command will respond with the same number.

2.2.4 On-the-fly commands

On-the-fly commands allow to immediately change some EES configuration parameters without having to restart the device. The parameters are applied both on the hardware and on the current EES configuration stored in RAM. The parameters are not saved in the NVM, so after a cold reset or a power off/on cycle the configuration is lost.

2.2.4.1 Address Table

Remove all learned entries

URL/api/at/del_all_learned discards all MAC addresses learned by the switch so far, so the learning process will start from scratch.

Remove a concrete entry

URL/api/at/del_entry.msgpack removes the specified entry (either learned or static). If the entry is static, the entry will be removed from the EES configuration too. Within the request the MAC address and VLAN ID must be sent encoded using the msgpack format.
The msgpack data to send embedded in the POST request shall have the following format

```
{
        "mac": [10, 20, 30, 40, 50, 160],
        "vlan_id ": 100
}
```

- *msgpack is a binary format, so it cannot be represented in a text document like this one. To represent the format in this specification the Json format is used, but the data sent in the POST request must be encoded in msgpack format.*
- *All numbers shall be coded in decimal base.*

Add entry

URL/api/at/add_entry.msgpack adds a static entry in the switch and also in the EES configuration. Within the request, the MAC address and VLAN ID must be sent encoded using the msgpack format.
The msgpack data to send embedded in the POST request shall have the following format

```
{
        "mac": [1, 2, 3, 4, 5, 6],
        "vlan_id": 100
        "ports": [0, 1]
}
```

- *msgpack is a binary format, so it cannot be represented in a text document like this one. To represent the format in this specification the Json format is used, but the data sent in the POST request must be encoded in msgpack format.*
- *All numbers shall be coded in decimal base.*

Replace AT entry

URL/api/at/replace_entry.msgpack replaces a static entry with a new one.
The msgpack data to send embedded in the POST request shall have the following format

```
{
        "del": {
                "mac": [1, 2, 3, 4, 5, 6],
                "vlan_id": 100
                }
        "add": {
                "mac": [1, 2, 3, 4, 5, 6],
                "vlan_id": 101
                "ports": [0, 1]
                }
}
```

- *msgpack is a binary format, so it cannot be represented in a text document like this one. To represent the format in this specification the Json format is used, but the data sent in the POST request must be encoded in msgpack format.*
- *All numbers shall be coded in decimal base.*

Read first AT learned entries

URL/api/at/read_first_learned_entries.msgpack Read the first 400 entries learned from the ART Table.
The msgpack data response in the POST request will have the following format

```
{
          "entries": [
            {
              "mac": [0, 14, 198, 99, 22, 171],
              "vlan_id": 10,
              "ports": [0],
              "aging": 7
            },
            {
              "mac": [100, 114, 18, 45, 222, 170],
              "vlan_id": 100,
              "ports": [3],
              "aging": 7
            }
          ],
          "last": true
}
```

- *msgpack is a binary format, so it cannot be represented in a text document like this one. To represent the format in this specification the Json format is used, but the data received in the POST request will be encoded in msgpack format.*
- *All numbers are coded in decimal base.*

2.2.4.2 Ports

Set basic configuration

URL/api/port/X/set.msgpack allows to enable/disable a port and change some of the physical layer parameters: master/slave role, speed, and mode in case of SFP ports.

X is the port id [0 to 9]:

- *X* from 0 to 7 corresponds to 1000BASE-T1 to 1000BASE-T8
- *X* from 8 to 9 corresponds to SFP+A to SFP+B

The msgpack data to send embedded in the POST request shall have the following format

```
{
        "enable": true,
        "master": false,
        "speed" : 1000,
        "mode" : 0
}
```

The field `speed` can take the following values:

speed field	speed
0	AUTO (only for Base-T interface)
10	10Mbps (only for Base-T interface)
100	100Mbps
1000	1Gbps

The field `mode` can take the following values:

mode field	SFP configuration
0	10GBASE-R interface
1	USXGMII interface
2	SGMII interface
3	1000BASE-X interface
4	2500BASE-X interface
5	5GBASE-R interface

2.2.4.3 PTP reconfiguration

Reset the initial time of PTP grandmaster partitions

URL/api/ptp/origin_timestamp.msgpack allows to reset the time transmitted by the PTP partitions to the value specified in this command.
The msgpack data to send embedded in the POST request shall have the following format

```
[{
        "partition": 0,
        "init_time": 1716894119,
        "init_time_ns": 500000000
},
{
        "partition": 3,
        "init_time": 1716894119,
        "init_time_ns": 0
}]
```

- *msgpack is a binary format, so it cannot be represented in a text document like this one. To represent the format in this specification the Json format is used, but the data sent in the POST request must be encoded in msgpack format.*
- *All numbers shall be coded in decimal base.*
- *Partitions not set in the remote command are not modified.*

2.2.4.4 MKA keys

These remote commands are only available on MACsec compatible devices (TE-2003 and TE-2004)

Retrieve a MKA key

URL/api/mka/X/get_key.msgpack retrieves the MKA key stored in the position *X*

The msgpack data to send embedded in the POST request shall have the following format

```
{
  "label": "100",
  "cak": [ 16, 32, 48, 64, 16, 32, 48, 64, 16, 32, 48, 64, 16, 32, 48, 64 ],
  "ckn": [ 24, 132, 250, 45, 201 ],
  "ports": [ 0, 1 ]
}
```

- *msgpack is a binary format, so it cannot be represented in a text document like this one. To represent the format in this specification the Json format is used, but the data received in the POST request will be encoded in msgpack format.*
- *All numbers shall be coded in decimal base.*
- *'cak' field must be an array of 16 or 32 bytes.*
- *'ports' field code to which ports this key is assigned.*

Modify a MKA key

URL/api/mka/X/set_key.msgpack modifies the MKA key stored in the position *X*. The modification of the MKA key takes effect immediately, so there is no need to reboot the device.

The msgpack data to send embedded in the POST request shall have the following format

```
{
  "label": "100",
  "cak": [ 16, 32, 48, 64, 16, 32, 48, 64, 16, 32, 48, 64, 16, 32, 48, 64 ],
  "ckn": [ 24, 132, 250, 45, 201 ],
  "ports": [ 0, 1 ]
}
```

- *msgpack is a binary format, so it cannot be represented in a text document like this one. To represent the format in this specification the Json format is used, but the data sent in the POST request must be encoded in msgpack format.*
- *All numbers shall be coded in decimal base.*
- *'cak' field must be an array of 16 or 32 bytes.*
- *'ports' field code to which ports this key is assigned.*
- *Fields not present are not modified.*

3 Examples of use

The EES remote API is based on HTTP requests, so any tool that allows sending GET and POST requests can be used. GET requests can be generated directly from a web browser, so no further tools are needed to retrieve the data available through the remote command.
This section provides examples for accessing the remote API with standard tools such as the Linux curl command and python scripts.

3.1 Curl

The curl command is a tool used to transfer data from or to a server using various network protocols, including HTTP, HTTPS, FTP, and others. It is commonly used in command-line interfaces and scripts for tasks such as downloading files, sending data to a server, and testing APIs. The most relevant options for dealing with the EES remote API are:

- `-v` "optional" verbose mode, additional information about the request will be shown.
- `-o` specifies the file where the data read from the get http request will be saved.
- `-s` "optional" silent mode, no statistics info will be shown.
- `-d` provides the associated data in the body of the HTTP request. Force a POST request.
- `--data-binary @file_name` send the file content exactly with no extra processing whatsoever. When `--data-binary` is used curl will use a POST request.

3.1.1 Retrieve information

Read all current configuration data in msgpack and save it in a file

```
curl http://10.0.0.200/api/get/config.msgpack -o cfg.msgpack -s
```

Read all current configuration data in Json format and save it in a file

```
curl http://10.0.0.200/api/get/config.json -o cfg.json -s
```

Read part of current configuration data in Json format and save it in a file

For instance all parameters of the BaseT-1 port

```
curl http://10.0.0.200/api/get/config/ports/main/0 -o cfg.json -s
```

Read all current status data in Json format and save it in a file

```
curl http://10.0.0.200/api/get/status.json -o status.json -s
```

Check is HTTP requests can be accepted

The device shall respond with the same number sent into the POST request.

```
curl -d "01234567" http://10.0.0.200/api/cmd/alive -s
```

3.1.2 Set the configuration

Send the complete configuration and saves it in the device's NVM

The configuration will be effective once a power off/on cycle or a cold reset command is sent

```
curl --data-binary @cfg.msgpack \
http://10.0.0.200/api/set/strict/config_persistent.msgpack -s
```

Reset command to force a cold reset of the device and set the configuration stored in the Non-Volatile Memory of the device.

```
curl -d "" http://10.0.0.200/api/cmd/reset_cold -v
```

Send the complete configuration and saves it in the device's RAM

The configuration will be effective after a warm reset command is sent. This configuration will be lost in a power off/on cycle.

```
curl --data-binary @cfg.msgpack \
http://10.0.0.200/api/set/strict/config_non_persistent.msgpack -s
```

Reset command to force a warm reset of the device and set the current configuration stored in RAM.

```
curl -d "" http://10.0.0.200/api/cmd/reset_warm -v
```

Validate the configuration

Check if the sent configuration is correct, if not try to fix it if possible. The configuration is not stored neither in RAM nor in NVM.

```
curl --data-binary @cfg.msgpack \
http://10.0.0.200/api/validate/config.msgpack -o validate.msgpack -s
```

3.2 Python

To use the following python script, it is necessary to install the msgpack and requests modules.

```
import msgpack
import requests

FILE_PATH = 'cfg.msgpack'
IP_DST = '10.0.0.200'

URL_DEFAULT    = 'http://' + IP_DST + '/api/get/default_config.msgpack'
URL_VALIDATE   = 'http://' + IP_DST + '/api/validate/config.msgpack'
URL_UPLOAD     = 'http://' + IP_DST + '/api/set/strict/config_persistent.msgpack'
URL_RESET_COLD = 'http://' + IP_DST + '/api/cmd/reset_cold'

print("read default configuration")
response = requests.get(url=URL_DEFAULT)
msgpack_data = response.content
json_data = msgpack.unpackb(msgpack_data)
print(json_data)
input("Press Enter to continue...")

print("validate the configuration read previously")
response = requests.post(url=URL_VALIDATE, data=msgpack_data)
msgpack_data = response.content
json_data = msgpack.unpackb(msgpack_data)
```

```
print(json_data)
input("Press Enter to continue...")

print("write back the configuration")
response = requests.post(url=URL_UPLOAD, data=msgpack_data)
print("write strict: ", response.text)
input("Press Enter to continue...")

print("A reset is necessary for the new configuration to be activated")
response = requests.post(url=URL_RESET_COLD)
print("reset strict: ", response.text)
```

4 Changelog

Version	Chapter	Description	Date
0.1	All	First Draft	Sept 2021
0.2	All	Fix default configuration command	Oct 2021
0.3	All	Add new remote commands Add chapter with curl and python examples	May 2024

5 Contact

If you have any questions regarding this product, please feel free to contact us:

Technica Engineering GmbH
Leopoldstr. 236
80807 München
Germany

Technical support:
support@technica-engineering.de

General information:
Info@technica-engineering.de

Most current user manuals and product information:
https://technica-engineering.de/en/